

Multiple-Copy Group Data Sharing Scheme With Integrity Auditing Function for Consumer Electronics

Youtong Wang[✉], Jinyong Chang[✉], Ru Meng, Peiru Yang[✉], and Xia Zhou[✉]

Abstract—The proliferation of consumer electronics (CE), particularly IoT devices such as smart home hubs and in-vehicle systems, is generating massive amounts of user data, creating an urgent need for secure and manageable cloud storage solutions tailored for consumer technology environments. While cloud storage alleviates the burden on local devices and enables convenient data access, it introduces significant security concerns for consumer applications: potential data loss from cloud servers and unauthorized access during data sharing. Existing attribute-based encryption (ABE) implementations often suffer from low efficiency, which in turn leads to substantial storage overhead, while existing schemes seldom account for the organizational structures common in consumer scenarios—such as a family unit or a fleet of interconnected devices. This paper proposes an efficient data sharing mechanism with integrity auditing functionality for cloud storage. Specifically designed for such consumer-grade IoT environments, the mechanism combines flexible access control with multi-replica cloud storage to reduce the risk of data loss. We design a group manager (GM)-driven upload management subsystem to efficiently handle data uploads from organizational entities. Finally, security and performance analyses were conducted on the integrated system designed in this article. The results indicate that the proposed scheme has good security and strong performance advantages.

Index Terms—Consumer electronics, cloud storage, integrity auditing, attribute-based encryption, data sharing.

I. INTRODUCTION

WITH the rapid advancement of consumer electronics (CE), particularly the widespread adoption of consumer IoT devices [1] such as smart home hubs, connected vehicles, and wearable gadgets, an unprecedented volume of

personal and sensitive data is being continuously generated. This data explosion poses a critical challenge for CE systems: how to securely store, manage, and selectively share this data while ensuring its perpetual availability [2]. Cloud storage, as an online distributed storage paradigm, emerges as a pivotal enabler for managing this big data deluge in modern consumer technologies. It allows data owners (DOs) to outsource their storage burdens to cloud service provider (CSP), enabling ubiquitous access and facilitating data sharing among authorized users in typical CE environments, such as family members or collaborative user groups.

Despite its convenience, cloud storage introduces significant security and privacy concerns that are particularly acute in consumer electronics applications [3]. First, data loss remains a prevalent threat to CE users. For consumers, the loss of irreplaceable personal data (e.g., family photos, health records) from a CSP, even with financial compensation, is unacceptable [4]. Second, during data sharing in CE contexts, malicious users may attempt to gain unauthorized access to sensitive information [5]. While ciphertext-policy attribute-based encryption (CP-ABE) is a prominent technique for enforcing fine-grained access control, its direct application in consumer-grade electronics is hampered by low efficiency [6]. The substantial ciphertext expansion (by tens or even hundreds of times) inherent in many CP-ABE schemes leads to exorbitant storage costs for DOs, making it impractical for resource-constrained consumer devices. Third, in characteristic CE scenarios like smart homes or vehicle fleets, data originates not from a single user but from a group of entities (e.g., multiple family members' devices) [7]. Existing schemes often lack efficient mechanisms for a central entity (e.g., a family administrator) to manage data uploads and user membership within such consumer electronics ecosystems securely and efficiently.

To address data loss, integrity auditing protocols allow verifiers to check data integrity without possessing the entire file [8]. However, auditing alone cannot prevent data loss. Storing multiple replicas of data on the cloud is a more robust strategy, ensuring data recovery even if some copies are corrupted [9]. Nevertheless, naively combining these technologies—multi-copy storage, efficient group management, and fine-grained access control—results in a system with prohibitive computational and communication overhead for practical CE deployments [10]. The core research problem we identify is the lack of an integrated solution that efficiently

Received 30 October 2025; revised 17 January 2026; accepted 5 March 2026. Date of publication 9 March 2026; date of current version 28 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62476212, in part by the Key Research and Development Program Project of Shaanxi Province under Grant 2025CYBXM-007, in part by the Natural Science Foundation Project of Shaanxi Province under Grant 2025JCYBMS-774, and in part by the Open Project of ISN National Key Laboratory under Grant ISN25-02. (Corresponding authors: Jinyong Chang; Ru Meng.)

Youtong Wang and Jinyong Chang are with the School of Information and Control Engineering, Xi'an University of Architecture and Technology, Xi'an, Shaanxi 710055, China, and also with the State Key Laboratory of Integrated Services Networks, Xidian University, Xi'an, Shaanxi 710061, China (e-mail: changjinyong@xauat.edu.cn).

Ru Meng is with the College of Cyber Security, Jinan University, Guangzhou 510632, China (e-mail: mengru93@stu2021.jnu.edu.cn).

Peiru Yang and Xia Zhou are with the School of Information and Control Engineering, Xi'an University of Architecture and Technology, Xi'an, Shaanxi 710055, China.

Digital Object Identifier 10.1109/TCE.2026.3672187

1558-4127 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on September 10, 2026 at 07:44:50 UTC from IEEE Xplore. Restrictions apply.

combines multi-copy data availability, scalable group management, and lightweight fine-grained access control, specifically designed for the resource and usability constraints of consumer electronics environments [11].

To bridge this gap, this paper proposes a new scheme. Our work is motivated by real-world CE applications, such as a family cloud storage where a parent (acting as the group manager) manages and secures data uploaded by various family members' devices, sharing it selectively with other relatives under flexible policies. The contributions of this paper are summarized as follows:

- 1) We design a group manager (GM)-based data upload subsystem tailored for organizational structures common in CE (e.g., families, device fleets). This subsystem supports efficient user revocation, ensuring that only legitimate members can upload data.
- 2) We introduce a mechanism where the GM generates multiple copies of a file to ensure data availability against loss. A hybrid encryption approach is adopted, leveraging a symmetric key for bulk data and a specially designed CP-ABE for key distribution, which dramatically reduces ciphertext expansion compared to pure ABE solutions.
- 3) We implement an efficient integrity auditing scheme for each replica. The GM generates authentication tags before outsourcing, enabling a third-party auditor (TPA) to verify the integrity of all copies stored at the CSP without retrieving the entire files.
- 4) We conduct comprehensive security and performance analyses. The results demonstrate that our proposed scheme achieves robust security against malicious CSPs and unauthorized users while maintaining significant performance advantages suitable for consumer-grade applications.

Related Works. We discuss related works from the following three perspectives.

- (1) **Secure Data Sharing and Access Control: The Efficiency Bottleneck.** Secure data sharing serves as a fundamental driver for cloud storage adoption, enabling effective collaboration among data owners and authorized users. Initial research efforts primarily focused on enhancing security functionalities within data sharing paradigms. Representative works include Shen et al.'s traceable group data sharing scheme for malicious user accountability [12], and Ming et al.'s approach for sensitive information sanitization [13]. The prevailing methodology across these schemes involves encrypting data before sharing and distributing decryption keys exclusively to authorized entities. To achieve fine-grained and flexible access control over encrypted data, attribute-based encryption (ABE), particularly CP-ABE, has emerged as a powerful cryptographic primitive [14]. This technique enables data owners to define and embed access policies directly within ciphertexts. However, a significant efficiency bottleneck impedes its practical deployment in resource-constrained environments like consumer tech-

nologies: substantial computational overhead coupled with notable ciphertext expansion. This expansion, often increasing data size significantly, leads to prohibitive storage and communication costs for data owners.

Consequently, considerable research efforts have been directed toward improving ABE practicality. Li et al. [15] addressed the challenge of efficient user revocation by introducing user group structures into CP-ABE, achieving secure revocation while resisting collusion attacks. Similarly, Xu et al. [16] designed a scheme supporting dynamic users and fine-grained access control. To overcome other limitations, Lan et al. [17] proposed a novel data sharing scheme that effectively mitigates the key escrow problem inherent in certain ABE systems. Beyond basic access control functionalities, Femminella et al. [18] recognized the need for search capabilities and proposed an attribute-based searchable encryption (ABSE) scheme, enabling authorized keyword search over encrypted data.

It is worth noting that alternative cryptographic approaches, including identity-based privacy-preserving remote data integrity checking [19], also facilitate secure data sharing and verification. Nevertheless, these schemes typically prioritize integrity and identity management, while their access control mechanisms generally lack the expressiveness offered by CP-ABE. In summary, despite significant advances in functionality—including traceability, revocation, searchability, and access control—implementation efficiency remains a critical barrier. CP-ABE's computational intensity and ciphertext expansion particularly limit its suitability for resource-constrained consumer scenarios.

- (2) **Data Integrity Auditing: From Single-Copy to Multi-Copy.** Ensuring the integrity of remotely stored data represents a cornerstone of trustworthy cloud storage. The pioneering works in this domain introduced two fundamental concepts: provable data possession (PDP) [20], which enables integrity verification without retrieving entire files, and proof of retrievability (PoR) [21], which provides stronger guarantees by ensuring complete file recoverability and emphasizing data retrievability. In contrast to other schemes, Yan et al. [22] proposed an efficient remote data possession verification protocol based on homomorphic hash functions. This protocol significantly reduces the communication and computational overhead associated with supporting dynamic operations while ensuring security against forgery, replacement, and replay attacks. To alleviate the computational burden on individual users, the concept of TPA was introduced [23]. However, concerns regarding TPA trustworthiness prompted the development of schemes leveraging decentralized technologies. Blockchain and smart contract-based solutions [24] emerged to achieve trust-free public auditing and resist malicious auditors. Parallel research efforts focused on simplifying key management mechanisms. To address the complexities associated with traditional public key infrastructure

(PKI), identity-based and certificateless audit schemes have been proposed [25]. Among these, Li et al. [26] introduced an identity-based privacy-preserving remote data integrity checking scheme to mitigate the risk of data privacy leakage during the auditing process. This scheme employs homomorphic verifiable tags to reduce system complexity, and by adding a random integer mask to the original data in the proof, it achieves audit privacy while simplifying certificate management.

To address the risk of data loss in single-replica storage, the research paradigm has shifted to multi-replica cloud storage [27]. In this direction, Li et al. [29] proposed an efficient identity-based provable multi-replica data possession scheme for multi-cloud environments. It is recognized as the first identity-based PDP scheme for multi-cloud and multi-replica scenarios, effectively enhancing data availability and audit efficiency.

However, this scheme, along with subsequent improvements, focuses primarily on the core design of multi-replica possession proofs and cross-cloud collaboration. It does not deeply address the needs for dynamic membership coordination by group managers and fine-grained attribute-based access control in consumer scenarios such as smart homes [28]. This leads to the next critical challenge: how to construct a comprehensive framework for multi-replica data that integrates efficient group management with flexible access control.

- (3) **Integrated Schemes for Group and Multi-Copy Scenarios.** Recognizing the practical necessities of organizational data management, several research works have begun integrating the GM concept into cloud storage architectures. These endeavors represent the state-of-the-art most closely aligned with our proposed work, as they attempt to concurrently address multiple challenges.

Tian et al. [31] pioneered an identity-based multi-copy data sharing auditing scheme employing a decentralized trust management architecture. This scheme delegates the core responsibilities of replica generation and signature creation to the GM, which aligns well with centralized management requirements in scenarios such as smart homes. However, in-depth analysis reveals that its trust management mechanism lacks a concrete implementation plan, and the advocated replica compression method fails to effectively address fundamental issues, consequently limiting its practical application value.

To more thoroughly resolve trust concerns, Liu et al. [32] proposed a blockchain-based integrity auditing scheme for group-shared data. This scheme fully leverages the inherent characteristics of blockchain to achieve transparent and tamper-proof user trust management. However, further investigation reveals that the GM's centralized key generation mechanism incurs significant computational overhead during user revocation, a design flaw that makes the system unsuitable for dynamic group environments.

Guo et al. [33] made significant progress by proposing a revocable and dynamic identity-based multi-copy data auditing scheme designed for multi-cloud environments. Its core innovation lies in a novel user key generation strategy that

enables efficient user revocation while supporting identity tracking, effectively avoiding the heavy revocation overhead present in schemes like [32]. However, the method of generating multiple copies on the data owner side increases the computational and communication costs for data owners.

A. Bilinear Map

Suppose $\mathbb{G}$ and $\mathbb{G}_1$ are two multiplicative cyclic groups of large prime q , and let g be a generator of $\mathbb{G}$. $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_1$ is a bilinear map with the following properties:

- 1) **Computability.** For any legitimate input $u, v \in \mathbb{G}$, it is efficient to compute $e(u, v)$.
- 2) **Bilinearity.** Once valid $a, b \in \mathbb{Z}_q^*$, $u, v \in \mathbb{G}$ has been provided, there is $e(u^a, v^b) = e(u, v)^{ab}$.
- 3) **Non-Degeneracy.** For any given $u, v \in \mathbb{G}$, there exists $e(u, v) \neq 1_{\mathbb{G}_1}$.

B. Security Assumptions

Let $\mathbb{G}$ and $\mathbb{G}_1$ be two multiplicative cyclic groups of prime q , and g is the generating element of $\mathbb{G}$. Let $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_1$ be a bilinear map.

- 1) **Discrete Logarithm (DL) Assumption.** Randomly select an element $a \in \mathbb{Z}_q^*$ and compute g^a . For a given g, g^a , any probabilistic polynomial-time (PPT) algorithm cannot compute the element a .
- 2) **Decisional Bilinear Diffie-Hellman (DBDH) Assumption.** Randomly select three elements $a, b, c \in \mathbb{Z}_q^*$. The DBDH assumption means that, given g, g^a, g^b, g^c , it is not feasible to calculate $L = e(g, g)^{abc}$, $L \in \mathbb{G}_1$.

C. Access Structure

Let $\{P_1, \dots, P_n\}$ denote the set of parties. Given a set $\mathbb{A} \subseteq 2^{\{P_1, \dots, P_n\}}$, select any element B and a set C such that $B \in C$. If $C \subseteq \mathbb{A}$ is satisfied, the $\mathbb{A}$ is monotone. The access structure is a monotone set $\mathbb{A} \subseteq 2^{\{P_1, \dots, P_n\}} \setminus \{\emptyset\}$ consisting of non-empty subsets of $\{P_1, \dots, P_n\}$. Those in $\mathbb{A}$ are called authorised sets; otherwise, they are called unauthorised sets.

D. Symmetric Encryption Scheme

Symmetric encryption algorithm means that encryption and decryption use the same key value, which requires the sender and receiver to negotiate a key before secure communication.

- 1) $\text{SymKeyGen}(1^k) \rightarrow K$. It generates key value K based on security parameter 1^k .
- 2) $\text{SymEnc}(K, M_1) \rightarrow M_1^*$. DO uses the key K to encrypt the data M_1 , generating the ciphertext M_1^* .
- 3) $\text{SymDec}(K, M_1^*) \rightarrow M_1$. DU decrypts the ciphertext M_1^* with the key K to obtain the shared data M_1 .

E. Asymmetric Encryption Scheme

Asymmetric encryption, also known as public key encryption, means that encryption and decryption use different key values. Suppose two users want to encrypt and exchange data, and both parties exchange public keys.

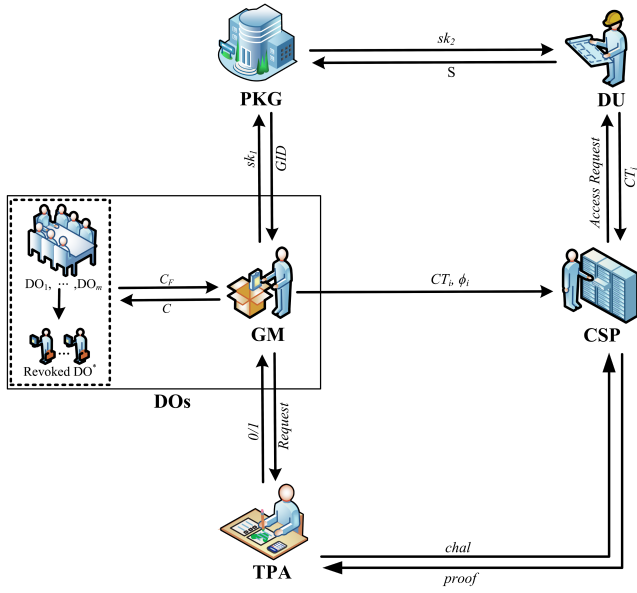


Fig. 1. System model of our scheme.

- 1) $\text{AsyKeyGen}(1^k) \rightarrow (sk, pk)$. It generates DU's public-secret key pairs (sk, pk) according to the security parameter 1^k .
- 2) $\text{AsyEnc}(pk, M_2) \rightarrow M_2^*$. DO uses the public key pk of the DU to encrypt the shared data M_2 to generate ciphertext M_2^* .
- 3) $\text{AsyDec}(sk, M_2^*) \rightarrow M_2$. DU uses own private key sk to decrypt the ciphertext M_2^* to obtain data M_2 .

II. SYSTEM AND SECURITY MODELS

A. System Model

Our proposed system consists of five kinds of entities: private key generator (PKG), data owners (DOs), cloud service provider (CSP), third-party auditor (TPA), and data user (DU). The relationships between them are shown in Fig. 1.

- 1) PKG: Private key generator is a fully-trusted entity who is responsible for generating group manager's private key and data user's attribute private key.
- 2) DOs: Data Owners are legitimate members within organizational settings, such as family members in a smart home. A GM, designated from among the DOs, oversees group operations including data processing and key management. For clarity in subsequent descriptions, the GM is treated as a separate logical entity. The subsequent DOs set includes revoked DO^* and active $\text{DO}_1 \dots \text{DO}_m$.
- 3) CSP: The cloud service provider is a semi-honest entity. The main service provided to GM is data storage.
- 4) TPA: Third party auditor helps GM determine the integrity of the data stored on CSP by generating challenge messages and verifying the returned proofs.
- 5) DU: Data users are authorized individuals such as family members accessing shared family data or organizational personnel accessing corporate data, who attempt to access the stored data managed by the GM.

The algorithms included in each stage are as follows.

(1) System Initialization.

- 1) $\text{Setup}(1^k) \rightarrow (params, msk)$. This algorithm is executed by PKG. It generates system public parameters $params$ and the master secret key msk , based on the security parameter 1^k . The parameters $params$ are public and implicitly included in the following algorithms. The master secret key msk is secretly kept by the PKG.
- 2) $\text{KeyGen}(msk, GID)$ or $(msk, S) \rightarrow (sk_1 \text{ or } sk_2)$. The algorithm is run by PKG to generate a secret key for GM or DU. If the input is an identity GID of GM, then compute and output the secret key sk_1 . Else if the input is the attribute set S , then output the attribute-based decryption secret key sk_2 for DU.
- 3) $\text{KeyAgr}(GID, \{ID_1, \dots, ID_m\}) \rightarrow K_1$. This is a sub-protocol between GM and valid DOs with identities $ID_1, \dots, ID_m$, which is to generate a common key K_1 .
- 4) $\text{KeyUpdate}(GID, \{ID_1, \dots, ID_m\}) \rightarrow \bar{K}_1$. This is a sub-protocol to update the common key K_1 as a new one $\bar{K}_1$ if some data owners need to be removed from the group.

(2) Upload Phase.

- 1) $\text{UpFile}(GID, ID_i, K_1, F)$: This is a sub-protocol between GM and the i th DO, which implements the function of transmitting the data file F from DO_i to GM based on the common key K_1 .
- 2) $\text{RepGen}(F) \rightarrow (\{F_i\})$. This algorithm is run by GM to generate multiple copies of F .
- 3) $\text{Encrypt}(F_i, \mathbb{T}) \rightarrow CT_i$. This algorithm is run by GM to generate an attribute-based ciphertext. That is, given the file F_i and access policy $\mathbb{T}$, it computes and outputs a ciphertext CT_i .
- 4) $\text{SigGen}(sk_1, CT_i) \rightarrow \phi_i$. This algorithm is still run by GM to generate the authentication tag ϕ_i , where the inputs consist of signing key sk_1 and the ciphertext CT_i . GM transmits (CT_i, ϕ_i) to CSP for storage.

(3) Audit Phase.

- 1) $\text{ChalGen}(\tau) \rightarrow chal$. This algorithm is performed by the TPA. To check the integrity of the data stored in CSP, it generates a challenge message $chal$.
- 2) $\text{ProofGen}(chal, CT_i, \phi_i) \rightarrow proof$. This algorithm is run by CSP. Based on $chal$, it generates the corresponding proof $proof$ and returns it to the TPA.
- 3) $\text{Verify}(chal, proof) \rightarrow (1/0)$. This algorithm is run by TPA, who verifies the validity of $proof$.

(4) Data Sharing Phase.

- 1) $\text{Decrypt}(sk_2, CT_i) \rightarrow F_i$. This algorithm is run by DU, which will decrypt the ciphertext CT_i as the file F_i .
- 2) $\text{DeRandom}(F_i) \rightarrow F$. This algorithm is used by DU to recover the original data file F .

B. Du Revocation Policy

The DU revocation policy based on CP-ABE adopts a forward-looking attribute management approach. Its core advantage lies in the zero overhead during normal system operation—routine revocations do not trigger any cryptographic operations and only involve updates to the policy logic maintained locally by the GM. This design explicitly accepts the trade-off of non-immediate invalidation of access rights to historical ciphertexts in exchange for lossless daily operations. For special cases requiring immediate revocation, an on-demand proactive re-encryption operation (a single CP-ABE encryption) can be triggered.

This strategy shifts the focus of security management from controlling the past to governing the future. For consumer electronics scenarios where data is highly time-sensitive and undergoes frequent version updates (e.g., family photos, dashcam footage), this represents an efficient and pragmatic engineering trade-off.

C. Security Model

According to the proposed protocol, we consider two types of adversaries: a malicious CSP and unauthorized DU. In order to achieve integrity auditing, after the GM uploads the data to the CSP, any malicious CSP that loses the data cannot generate a forged proof that can be verified by the TPA. In order to achieve confidentiality of shared data, any malicious DU, after obtaining encrypted data from the CSP, will not be able to decrypt the ciphertext to obtain original data without access privileges.

Based on these two threats, we characterize the security model through the following two games.

Game 1: We suppose a security game 1 between a challenger C_1 and a malicious adversary A_1 to describe a security model for storage integrity of cloud data. In game 1, it is assumed that the PKG, GM, and TPA are fully trusted. The adversary A_1 simulates a dishonest CSP whose objective is to forge valid audit proof.

- 1) **Initialization.** C_1 executes *Setup* to generate msk and $params$, and sends $params$ to A_1 .
- 2) **Query Phase.** The following queries are allowed for A_1 .
 - a) **Hash_Query:** A_1 can query the hash values corresponding to the hash functions H, H_1 and H_2 in the $params$ based on the selected data.
 - b) **Secret_Key_Query:** A_1 sends selected group identity to C_1 . C_1 performs *KeyGen* to generate private key and sends it to A_1 .
 - c) **Sign_Query:** A_1 can query the corresponding signature for the adaptively selected data block.
- 3) **Challenge Phase.** C_1 launches a challenge to A_1 , who is required to return a corresponding proof.
- 4) **Forgery Phase.** A_1 forges proof based on challenge and sends it to C_1 . If it passes verification, A_1 wins.

Definition 1: The scheme is safe against malicious CSP, if the probability of any PPT adversary A_1 winning in Game 1 is negligible.

Game 2: We assume a security game 2 with a challenger C_2 and a malicious adversary A_2 to characterize the

security model of indistinguishability under a chosen plaintext attack. The adversary A_2 simulates an unauthorized DU.

- 1) **Initialization.** It follows the same process as initialization in Game 1.
- 2) **Query Phase 1.** The A_2 issues a query request to the challenger C_2 to obtain the attribute private key associated with a specific attribute set.
- 3) **Challenge Phase.** The A_2 randomly selects two plaintext messages M_0 and M_1 . After comparing the attribute set with the access control tree, the A_2 identifies incompatibility. Subsequently, A_2 sends the chosen plaintexts and the access control tree to the C_2 . Upon receipt, C_2 randomly selects a $b \in \{0, 1\}$, encrypts the plaintext using a predefined access policy, and finally transmits the ciphertext to A_2 .
- 4) **Query Phase 2.** A_2 sends another query request to the C_2 , demanding the attribute private key associated with a specific attribute set. However, the queried attribute set still fails to satisfy the access control tree.
- 5) **Forgery Phase.** A_2 guesses $b \in \{0, 1\}$. If $b' = b$, then A_2 wins.

Definition 2: The scheme is secure under the choice of plaintext attack, if the probability $P_r[b' = b] - \frac{1}{2}$ that A_2 wins game 2 is negligible.

III. OUR PROPOSED CONSTRUCTION

The specific details of each stage are as follows.

(1) System Initialization.

- 1) *Setup*(1^k) $\rightarrow$ ($params, msk$). Given the security parameter 1^k as input, PKG first determines two multiplicative cyclic groups $\mathbb{G}$ and $\mathbb{G}_1$ with the order q . g is generator of $\mathbb{G}$ and $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_1$ is a bilinear mapping. $H, H_1 : \{0, 1\}^* \rightarrow \mathbb{G}$, $H_2 : \mathbb{G} \times \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ are three hash functions. PKG randomly selects two elements $x, y \in \mathbb{Z}_q^*$ and defines $msk = (x, y)$ as the master key. Compute $Y_1 = e(g, g)^x$, $Y_2 = g^y$, and define the system public parameters as $params = (\mathbb{G}, \mathbb{G}_1, q, g, e, H, H_1, H_2, Y_1, Y_2)$.
- 2) *KeyGen*(msk, GID) or (msk, S) $\rightarrow$ (sk_1 or sk_2). If the input is the group identity GID , then this algorithm computes the private key $sk_1 = H(GID)^y$ as its signing key. Else if the input is DU's attribute set S , then randomly choose $r_j \in \mathbb{Z}_q^*$, for $j \in S$, and an additional element $r \in \mathbb{Z}_q^*$. The attribute-based decryption secret key sk_2 is calculated as follows.

$$sk_2 = \begin{cases} D = g^{\frac{x+r}{y}}; \\ \forall j \in S : D_j = g^{r_j} \cdot H_1(j)^{r_j}, D'_j = g^{r_j}. \end{cases}$$

- 3) *KeyAgr*($GID, \{ID_1, \dots, ID_m\}$) $\rightarrow K_1$. In this sub-protocol, GM first generates a key K_1 of symmetric encryption scheme (*SymKeyGen*, *SymEnc*, *SymDec*), and each DO_i with identity ID_i computes public-secret key pairs (sk^i, pk^i) based on an asymmetric encryption scheme (*AsyKeyGen*, *AsyEnc*, *AsyDec*). Then GM computes $AsyEnc(pk^i, K_1) \rightarrow C_{ID_i}$ and gives it

to DO_i , who can decrypt C into K_1 based on its secret key sk^i .

- 4) $KeyUpdate\langle GID, \{ID_1, \dots, ID_m\} \rangle \rightarrow \bar{K}_1$. When it is necessary to remove DO^* from the group, the GM generates a new key $SymKeyGen(1^k) \rightarrow \bar{K}_1$. The GM then encrypts the key $AsyEnc(pk^j, \bar{K}_1) \rightarrow C_{ID_j}$ using the public key pk^j of member DO_j and sends it to the respective member. DO_j decrypts $AsyDec(sk^j, C_{ID_j}) \rightarrow \bar{K}_1$. Thereafter, all legitimate members DO_j must use the new key $\bar{K}_1$ to encrypt their data files in the *UpFile* protocol, whereas any revoked member DO^* is unable to obtain $\bar{K}_1$.

(2) Upload Phase.

- 1) $UpFile\langle GID, ID_i, K_1, F \rangle$: For the data file F , the i -th valid DO first encrypts it into a ciphertext by running $SymEnc(K_1, F) \rightarrow C_F$ and sends it to GM, who can decrypt C_F by running $SymDec(K_1, C_F) \rightarrow F$.
- 2) $RepGen(F) \rightarrow \{F_i\}$. In order to generate multiple copies of file F , GM first divides the F into n parts $\{m_j \in Z_q^* \mid j = 1, \dots, n\}$. For each $1 \leq j \leq n$, compute $m_{ij} = m_j + H_2(fname || i || j)$, where $fname$ is the filename of original data file F . The i th copy is $F_i = \{m_{i1}, \dots, m_{in}\}$.
- 3) $Encrypt(F_i, \mathbb{T}) \rightarrow CT_i$. Given the copy file F_i , this algorithm first generates a symmetric encryption key K_2 by running $SymKeyGen(1^k) \rightarrow K_2$, and encrypts F_i into $C_1^i = SymEnc(K_2, F_i)$. Then encrypt K_2 based on a CP-ABE scheme. More concretely, randomly choose a number $s \in Z_q^*$ as the secret value, and construct the access tree $\mathbb{T}$ as follows: Suppose Y is the set of leaf nodes, $parent(a)$ is the parent of node a in the $\mathbb{T}$, $attr(a)$ is the attribute associated with the leaf node a , and $index(a)$ is the index of the child node of a . Let the threshold value of node a be b . Choose a d_a nd degree polynomial for node a , $d_a = b - 1$. Let R be the root node, randomly choose the d_R th polynomial $q_R(a)$ to satisfy $q_R(0) = s$, and for the other nodes a' , construct the $d_{a'}$ th polynomial in the same way to satisfy

$$q_{a'}(0) = q_{parent(a')}(index(a')).$$

Then, this algorithm encrypts K_2 into C_2^i :

$$C_2^i = \begin{cases} Y; \\ C_{K_2} = K_2 \cdot e(g, g)^{xs}, C = g^{ys}; \\ \forall t \in Y, C_t = g^{q_t(0)}, C'_t = H_1(attr(t))^{q_t(0)}. \end{cases}$$

Finally, the ciphertext CT_i is $(fname, i, \mathbb{T}, C_1^i, C_2^i)$.

- 4) $SigGen(sk_1, CT_i) \rightarrow \phi_i$. First, parse ciphertext CT_i as $(fname, i, \mathbb{T}, C_1^i, C_2^i)$ and define C_1^i as $\{c_{i1}, c_{i2}, \dots, c_{in}\}$, $c_{in} \in Z_q^*$. Then randomly choose a $\xi \in Z_q^*$ and compute $Y_3 = g^\xi$. For $1 \leq j \leq n$, compute the tag

$$\sigma_{ij} = sk_1 \cdot (H_1(fname || i || j)g^{c_{ij}})^\xi.$$

Define the authentication tag as $\phi_i = (Y_3, \sigma_{i1}, \sigma_{i2}, \dots, \sigma_{in})$. Finally, GM transmits (CT_i, ϕ_i) to CSP for storing.

(3) Audit Phase.

- 1) $ChalGen(\tau) \rightarrow chal$. Given a $\tau \in [n]$, this algorithm randomly selects τ different positions of data blocks and coefficients, each of which has the form of $(j, v_j) \in [n] \times Z_q^*$. Output the challenge message $chal = \{(j, v_j)\}$.
- 2) $ProofGen(chal, CT_i, \phi_i) \rightarrow proof$. After receiving $chal$ from TPA, this algorithm aggregates the stored data blocks and their tags as follows.

$$\mu = \sum_j c_{ij} \cdot v_j, \omega = \prod_j \sigma_{ij}^{v_j}.$$

Finally CSP returns $proof = (\mu, \omega, Y_3, fname, i)$ to the TPA.

- 3) $Verify(chal, proof) \rightarrow (1/0)$. Once the TPA receives $proof$ from CSP, it checks the validity of this proof by computing

$$e(\omega, g) = e\left(H(GID)^{\sum_{j \in chal} v_j}, Y_2\right) \cdot e\left(\left(\prod_{j \in chal} H_1(fname || i || j)^{v_j}\right) g^\mu, Y_3\right). \quad (1)$$

If it is correct, output 1; Else output 0.

(4) Data Sharing Phase.

- 1) $Decrypt(sk_2, CT_i) \rightarrow F_i$. After receiving the ciphertext CT_i , the DU decrypts it as follows. Let z be a child of a and define the recursive function:

$$DecptNode(sk_2, CT_i, a), u = attr(a).$$

- a) If a is a leaf node, DU executes function:

$$F_a = DecptNode(sk_2, CT_i, a), v = attr(a).$$

If $v \in S$, then there exists:

$$F_a = e(g, g)^{rq_a(0)}. \quad (2)$$

- b) If a isn't a leaf node, DU executes function:

$$F_z = DecptNode(sk_2, CT_i, z).$$

Denote by S_a the set of arbitrary children of node a such that $F_z \neq \perp$. If no such set S_a exists, then the node a is unsatisfiable and returns $\perp$. If

$$S'_a = \{index(z), z \in S_a\}, i = index(z),$$

we have: $F_a = e(g, g)^{rs}$.

If the attributes of DU satisfy the access policy $\mathbb{T}$, F_R can be obtained from $DecNode(sk_2, CT_i, R)$:

$$F_R = e(g, g)^{rq_R(0)} = e(g, g)^{rs}.$$

Then K_2 can be calculated as

$$\frac{C_{K_2} \cdot F_R}{e(C, D)} = K_2. \quad (3)$$

If the attributes of DU do not satisfy $\mathbb{T}$, then he cannot obtain K_2 . Otherwise, he can obtain the decryption key K_2 for the ciphertext C_1^i . Based on this key K_2 , DU can get F_i by running $\text{SymDec}(K_2, C_1^i)$.

- 2) $\text{DeRandom}(F_i) \rightarrow F$. As last, when DU obtains F_i , DU parses the file F_i as $\{m_{i1}, m_{i2}, \dots, m_{in}\}$, and recovers the original file $F = \{m_1, m_2, \dots, m_n\}$ that DO wants to share with him by computing $m_j = m_{ij} - H_2(\text{fname} \| i \| j)$, for $1 \leq j \leq n$.

IV. CORRECTNESS AND SECURITY ANALYSIS

A. Correctness Analysis

In the *SigGen* phase, if the GM uploads valid C_i^1 and the corresponding ϕ_i , it will pass the CSP verification. In the *Verify* phase, if C_i^1 stored by the CSP is complete, it will pass the TPA integrity verification. In the *Decrypt* phase, if the DU's sk_2 satisfies the $\mathbb{T}$, the DU can get K_2 , and then decrypt C_1^i to get F_i by K_2 .

There is a following note: $H' = H_1(\text{fname} \| i \| j)$. The specific process is described below:

- 1) Correctness of the integrity auditing verification. The procedure for verifying (1) is as follows:

$$\begin{aligned} e(\omega, g) &= e\left(\prod_{j \in \text{chal}} \sigma_{ij}^{v_j}, g\right) \\ &= e\left(\prod_{j \in \text{chal}} \left(sk_1 (H' g^{c_{ij}})^{\xi}\right)^{v_j}, g\right) \\ &= e\left(\prod_{j \in \text{chal}} (sk_1)^{v_j}, g\right) \\ &\quad \cdot e\left(\prod_{j \in \text{chal}} \left((H' g^{c_{ij}})^{\xi}\right)^{v_j}, g\right) \\ &= e\left(H(GID)^{\sum_{j \in \text{chal}} v_j}, Y_2\right) \\ &\quad \cdot e\left(\left(\prod_{j \in \text{chal}} H^{v_j}\right) g^{\mu}, Y_3\right). \end{aligned}$$

- 2) Correctness of access control validation.

- a) The procedure for verifying (2) is as follows:

$$\begin{aligned} F_a &= \frac{e(D_v, C_a)}{e(C'_v, D'_a)} \\ &= \frac{e(g^r \cdot H_1(v)^{r_v}, g^{q_a(0)})}{e(g^{r_v}, H_1(v)^{q_a(0)})} \\ &= e(g, g)^{r q_a(0)}. \end{aligned}$$

- b) The procedure for verifying (3) is as follows:

$$\begin{aligned} \frac{C_{K_2} \cdot F_R}{e(C, D)} &= \frac{K_2 \cdot e(g, g)^{xs} \cdot e(g, g)^{rs}}{e\left(g^{ys}, g^{\frac{x+r}{y}}\right)} \\ &= \frac{K_2 \cdot e(g, g)^{xs} \cdot e(g, g)^{rs}}{e(g, g)^{xs} \cdot e(g, g)^{rs}} \\ &= K_2. \end{aligned}$$

B. Security Analysis

We will prove the security of this scheme for A_1 under the CDH assumption and for A_2 under the DBDH assumption.

Theorem 1: If the CDH assumption holds, this scheme is resistant to malicious CSP, it can be realized that if a CSP stores an incomplete block of data, it is computationally incapable of generating a proof *proof* that can be verified by TPA.

Proof. Assume that A_1 can win Game 1 with non-negligible probability and construct a C_1 to try to solve the CDH assumption. Given (g, g^a, g^b) , its goal is to compute g^{ab} . A_1 interacts with C_1 as follows:

- 1) **Initialization.** C_1 firstly executes algorithm $\text{Setup}(1^k)$, gets $\text{params} = (q, g, e, Y_1, Y_2, H, H_1, H_2, \mathbb{G}, \mathbb{G}_1)$ and $\text{mpk} = g^a$, and sends params to A_1 .
- 2) **Queries.** A_1 can execute any of the following queries.

- a) **Hash-Query.** A_1 can choose any GID to run Hash-Query algorithm. C_1 maintains a list $L_H = (GID, H(GID), T, h)$ for query request GID^* . If the $GID^* \in L_H$, C_1 sends the corresponding tuple $(GID^*, h^*, T^*, H(GID^*))$ to A_1 . Otherwise, C_1 randomly chooses two values h^* and $T^* \in \{0, 1\}$. Assume that the probability of $T^* = 0$ is α , and the probability of $T^* = 1$ is $1 - \alpha$. If $T^* = 0$, C_1 computes $H(GID^*) = g^{h^*}$. Otherwise, C_1 computes $H(GID^*) = (g^b)^{h^*} = (g^{h^*})^b$. Finally, C_1 sends $H(GID^*) = g^{h^*}$ or $H(GID^*) = (g^b)^{h^*}$ to A_1 , and inserts the tuple $(GID^*, h^*, T^*, H(GID^*))$ in L_H .
- b) **Secret-Key-Query.** A_1 chooses any GID^* to run the algorithm. C_1 maintains a list L_{SK} for the $GID \in L_{SK}$. If $GID^* \in L_{SK}$, C_1 computes $H(GID^*)$ through Hash-Query. Otherwise, C_1 chooses the tuple $(GID^*, h^*, T^*, H(GID^*))$ in L_H and obtains T^* . If $T^* = 1$, C_1 terminates the algorithm. Otherwise, C_1 selects the tuple $(GID^*, h^*, T^*, H(GID^*))$ in L_H , gets the values of h^* and $H(GID^*)$, and performs the following procedure:
 - i) If $GID^* \notin L_{SK}$, C_1 computes $sk_1^* = H(GID^*)^a = (g^{h^*})^a$ or $sk_1^* = H(GID^*)^a = ((g^{h^*})^b)^a$. C_1 sends the tuple (GID^*, sk_1^*) to A_1 and inserts it into L_{SK} .
 - ii) If $GID^* \in L_{SK}$, C_1 selects the tuple (GID^*, sk_1^*) in L_{SK} , and sends sk_1^* to A_1 .
- c) **Sign-Query.** A_1 chooses any tuple (c_{ij}^*, GID^*) to request corresponding signature σ_{ij}^* from C_1 . C_1 maintains a list L_{Sig} for the query tuple (c_{ij}^*, GID^*) . First, C_1 computes the value $H(GID^*)$ corresponding to the GID^* if $GID^* \notin L_H$, otherwise, C_1 selects a corresponding tuple $(GID^*, h^*, T^*, H(GID^*))$ from L_H . If $T^* = 1$ in tuple, the algorithm is terminated. When $T^* = 0$ and $GID^* \notin L_{SK}$, C_1 computes the key sk_1^* corresponding to GID^* by Secret-Key-Query. If $GID^* \in L_{SK}$, then C_1 selects the tuple (GID^*, sk_1^*) and obtains the key $sk_1^* = (g^a)^{h^*}$ from it, and then continues with the following steps:

- i) If $(c_{ij}^*, GID^*) \notin L_{Sig}$, C_1 first selects a random value $\xi^* \in Z_q^*$ to generate a new signature $\sigma_{ij}^* = sk_1^*(H_1(fname^* || i || j)g^{c_{ij}^*})^{\xi^*}$ for a known data block $c_{ij}^* \in fname^*$. C_1 computes an auxiliary label $Y_3^* = g^{\xi^*}$ and then adds the tuple (c_{ij}^*, GID^*) to the list L_{Sig} and sends σ_{ij}^* to A_1 .
- ii) If $(c_{ij}^*, GID^*) \in L_{Sig}$, C_1 sends σ_{ij}^* of the tuple $(c_{ij}^*, GID^*, \sigma^*)$ in list L_{Sig} to A_1 .

3) **Forgery.** For a tuple $(c_{ij}, GID, \sigma_{ij}, Y_3)$, C_1 chooses a tuple $(GID, h, T, H(GID))$ in L_H . There is a following note: $H' = H_1(fname^* || i || j)$. A_1 wins if $T = 1$ and satisfies equation: $e(\sigma_{ij}^*, g) = e(H(GID^*), mpk) \cdot e(H'g^{c_{ij}^*}, Y_3^*)$.

From the output of A_1 , C_1 can calculate g^{ab} . We denote the probability that A_1 wins the game at time t after executing the query process *Queries* as ϵ , no more than n_h , n_{sk} , and n_{sig} times, respectively. When A_1 wins the game, the challenger C_1 does not terminate the query process, we can know that *Queries* have all been executed successfully. C_1 will terminate the query process with probability greater than $(1 - \alpha)^{n_{sk}n_{sig}}$ in Sign-Query and Secret-Key-Query. C_1 can obtain the correct result for the CDH problem with probability greater than $\epsilon^* \geq \epsilon\alpha(1 - \alpha)^{n_{sk}n_{sig}} \geq \frac{\epsilon}{(n_{sk} + n_{sig})2e}$ in less than $O(n_h + n_{sk} + n_{sig})$ time, which we know is impossible. This also ends the proof of Theorem 1.

Theorem 2: If the DBDH assumption holds, then this scheme is resistant to malicious DU. It means that DU do not get the key value K_2 without satisfying the access policy and thus cannot decrypt to get the original file F .

Proof. Assume that A_2 can win the game 2 with non-negligible probability, and construct a C_2 to try to solve the DBDH assumption. Given (g, g^a, g^b, g^c) , its goal is to compute g^{abc} . A_1 can execute any of the following queries.

- 1) **Initialization.** C_2 chooses three values a, b, c randomly and executes algorithm *Setup*(1^k) simultaneously to get *params*, and calculates $ab + \alpha = x$ to get the value of α .
- 2) **Query 1.** A_2 sends a query request to C_2 . After C_2 receives S^* , it computes the attribute private key sk_2^* and sends it to A_2 .
- 3) **Challenge.** A_2 obtains the access tree T^* , compares T^* with S^* , and finds that the condition is not satisfied. Then, A_2 sends the chosen plaintext M_0, M_1 and T^* to C_2 . C_2 generates a random number $b \in \{0, 1\}$, encrypts M_b , and sends the ciphertext to A_2 .
- 4) **Query 2.** A_2 obtains sk_2^* corresponding to a set S^* by repeating *Query 1*, but it still does not satisfy T^* .
- 5) **Forgery.** A_2 makes a guess at the ciphertext for choice b^* . C_2 makes a judgement on the choice of A_2 using the following process:

$$K_{b^*} \cdot e(g, g)^{xc} = K_{b^*} \cdot e(g, g)^{abc} \cdot e(g^a, g^c).$$

Thus, the probability that A_2 wins Game 2 is non-negligible. This ends the proof of Theorem 2.

C. Analysis of the Revocation Mechanism

In this section, the complexity and feasibility of the revocation mechanism will be analyzed. The revocation mechanism

of this scheme is designed with meticulous consideration for overhead control.

For the revocation of a DO, the computational overhead primarily consists of the GM performing m asymmetric encryptions, while each legitimate member performs one asymmetric decryption. The communication overhead scales linearly with the number of DO, with no additional storage cost incurred. In the case of DU revocation, the regular mode requires only a logical adjustment to the access policy, resulting in nearly negligible computational and communication overhead. It is only when a proactive re-encryption strategy is adopted that the scheme incurs the cost of one complete CP-ABE encryption operation and the communication overhead associated with transmitting the entire file, yet still without imposing additional storage requirements. By employing this lightweight, policy-driven revocation strategy, the scheme effectively circumvents these high instant overheads.

D. Trust Assumptions and Mitigations

Within the framework of standard cryptographic primitives adopted by this scheme, the PKG is defined as a fully trusted entity. This constitutes a foundational formal assumption for ensuring system confidentiality. If the PKG is compromised, all user private keys it has generated become invalid. Consequently, any access control and auditing mechanisms relying on these keys would be rendered meaningless. Future work may explore trust-distributed cryptographic architectures, such as multi-authority attribute-based encryption or threshold secret sharing techniques, to distribute the core functions traditionally held by a single PKG among multiple independent authorities.

This scheme designs the GM as the fully trusted operational core within the system, making it suitable for scenarios with clear managerial relationships, such as families or project teams. If the GM is fully compromised, it would lead to the failure of integrity guarantees, leakage of current group communications, and loss of management control. To reduce the absolute trust dependency on a single entity, a multi-administrator committee based on threshold cryptography could be introduced. By sharing authority and employing threshold mechanisms, the security foundation can be relaxed to the more robust assumption of a “majority honest” committee.

V. PERFORMANCE ANALYSIS

In this section, we will analyze the scheme from three aspects: functional analysis, theoretical analysis, and simulation experiment. The reason we choose these proposals is that they are all data integrity auditing schemes for GM, multi-copy or data sharing, which have similarities with our scheme.

A. Functional Analysis

As shown in Table I, our scheme supports all five features, making it more comprehensive and suitable for real-world consumer IoT scenarios such as smart homes and connected vehicle systems.

It is important to note that the current scheme is designed for static data. The current scheme’s limited support for dynamic

TABLE I
FUNCTION COMPARISON

Scheme	Sign Outsourcing	Multiple-Copy	Data Sharing	Privacy Protection	Access Control
Tian et al. [31]	Yes	Yes	Yes	No	No
Liu et al. [32]	No	No	No	Yes	No
Guo et al. [33]	No	Yes	Yes	No	No
Chen et al. [27]	No	Yes	Yes	No	No
Tian et al. [34]	No	Yes	No	Yes	No
Our scheme	Yes	Yes	Yes	Yes	Yes

TABLE II
SECURITY FUNCTION COMPARISON

Scheme	Collusion Resistance	Forward Secrecy	Malicious CSP Resistance	Privacy Preservation	Key Security
Tian et al. [31]	✓	×	✓	×	✓
Liu et al. [32]	✓	×	✓	✓	✓
Guo et al. [33]	✓	×	✓	×	✓
Chen et al. [27]	×	×	✓	×	✓
Tian et al. [34]	✓	×	✓	✓	✓
Our scheme	✓	✓	✓	✓	✓

TABLE III
COMMUNICATION COST COMPARISON

Scheme	DOs → GM	GM → CSP	TPA → CSP	CSP → TPA	DOs → CSP
Tian et al. [31]	$2(n q + G)$	$2n(q + G)$	$2\tau(n + q)$	$2(q + G)$	—
Liu et al. [32]	—	—	$\tau(n + q)$	$ q + G $	$n(q + G)$
Guo et al. [33]	—	—	$ n + q $	$ q + G $	$cn(q + G)$
Chen et al. [27]	—	—	$\tau(n + q)$	$ q + G $	$cn(q + G)$
Tian et al. [34]	—	—	$\tau n $	$ q +2 G $	$n(q + G)$
Our scheme	$n q + G $	$nc(q + G)+3(G + G_1)$	$\tau(n + q)$	$ q + G $	—

data operations stems from its core design choices optimized for static scenarios. The replica-tag coupling mechanism leads to cascading multi-replica overhead triggered by single-point updates; the tight binding of audit protocols to block indexes results in widespread tag invalidation when sequences are altered; and the coordination between fine-grained access control and dynamic updates introduces complexity in policy and key management. These are inherent trade-offs made to achieve efficient static management of multi-replica data and also make supporting efficient dynamic operations a clear direction for future research.

B. Security Function Comparison

In Table II, our scheme is the only one that ensures forward secrecy, achieved through the collaborative integration of key updates and policy adjustments. This characteristic is crucial for long-term data security in dynamic group settings. This ensures that even if a user's long-term private key is compromised in the future, it cannot be used to decrypt previously exchanged ciphertexts. Furthermore, unlike [27], our scheme provides robust collusion resistance, preventing multiple unauthorized users from pooling their attributes to gain access. It also incorporates privacy preservation, which is not considered in [31], [33], and [27], thereby effectively protecting user identity and access patterns from exposure to third parties, including the cloud server. This combination of features makes our scheme particularly suited for consumer-grade applications where user membership changes frequently and sensitive personal data is involved.

C. Theoretical Analysis

We compare the communication cost, computation cost and storage cost of our scheme with those of recent schemes. Some of the notations used are described in Table V.

CommunicationCost: We mainly consider the communication cost of the five processes. A comparison of the communication cost for each scheme at each stage is displayed in Table III.

In our comparison, we mainly focus on the key components of each phase in every scheme, such as signatures, while omitting some components with relatively low costs, like auxiliary tags. Our scheme reduces the communication overhead in the DOs→GM phase, and its overall advantage lies in lowering the communication cost on the side of DOs. This is crucial for resource-constrained consumer IoT devices with limited bandwidth. In the GM→CSP, the communication cost of our scheme is higher than that of the scheme in Tian et al.'s scheme [31]. The reason for this is that our scheme generates multiple copies of the file, whereas the scheme in [31] essentially has only one copy.

ComputationCost: By comparison, we find that we have almost the same computational cost as the other schemes in *ProofGen*, so it is not taken into account in Table IV of the computational cost comparison.

The analysis reveals that our approach streamlines the replica generation process, achieving a lower overhead than Tian et al. [31], and employs a significantly more efficient verification algorithm than Guo et al. [33], which suffers from

TABLE IV
COMPUTATION COST COMPARISON

Scheme	RepGen	SigGen	Verify
Tian et al. [31]	$cn(\mathcal{H} + \mathcal{A}_z) + no$	$2n(2\mathcal{E} + \mathcal{M} + \mathcal{H})$	$2\mathcal{P} + (\tau + 1)(\mathcal{H} + \mathcal{E}) + \varpi\mathcal{M}$
Liu et al. [32]	—	$n(\mathcal{M} + \mathcal{H} + 2\mathcal{E})$	$3\mathcal{P} + (\tau + 1)(\mathcal{E} + \mathcal{H}) + \tau\mathcal{M}$
Guo et al. [33]	$cn(\mathcal{A}_z + l)$	$nc(\mathcal{E} + \mathcal{M} + \mathcal{H}) + \mathcal{H} + \mathcal{E}$	$2\mathcal{P} + \tau(\mathcal{E} + 3\mathcal{H} + 3\mathcal{M}_z) + nct(\mathcal{M}_z + \mathcal{E})$
Chen et al. [27]	$cn\mathcal{K}$	$cn(2\mathcal{E} + \mathcal{M} + \mathcal{H}) + \mathcal{E}$	$2\mathcal{P} + c(\tau + 1)(\mathcal{E} + \mathcal{H})$
Tian et al. [34]	$cn(\mathcal{A}_z + l)$	$n(c + 1)(\mathcal{H} + 2\mathcal{E} + 2\mathcal{M})$	$3\mathcal{P} + c\tau(3\mathcal{E} + 2\mathcal{H} + \mathcal{M}) + 2\mathcal{M}$
Our scheme	$cn(\mathcal{A}_z + \mathcal{H})$	$cn(\mathcal{H} + 2\mathcal{E} + \mathcal{M})$	$2\mathcal{P} + (\tau - 1)(\mathcal{E} + \mathcal{H} + \mathcal{M}) + \mathcal{H}$

TABLE V
NOTATION IN THEORETICAL ANALYSES

Symbols	Definitions
$\mathcal{P}$	Bilinear pairing
$\mathcal{H}$	Hash computation
$\mathcal{M}_z, \mathcal{A}_z$	Multiplicative and Addition on $\mathbb{Z}_q^*$
o	Heterodyne operation
$\mathcal{K}$	Encryption Algorithm
$\mathcal{E}$	Square on $\mathbb{G}$ and $\mathbb{G}_1$
$\mathcal{M}$	Multiplicative on $\mathbb{G}$ and $\mathbb{G}_1$
$ q $	Length of elements $\mathbb{Z}_q^*$
l	function calculation
$ G , G_1 $	Length of elements $\mathbb{G}$ and $\mathbb{G}_1$

TABLE VI
STORAGE COST COMPARISON

Scheme	Storage Cost
Tian et al. [31]	$2n(q + G)$
Liu et al. [32]	$n(q + G)$
Guo et al. [33]	$n(c + 1)(q + G)$
Chen et al. [27]	$cn(q + G)$
Tian et al. [34]	$cn(q + G) + G $
Our scheme	$nc(q + G) + 3 G + G_1 $

prohibitive quadratic complexity. This results in a computed reduction in verification overhead, a critical enhancement for systems requiring frequent integrity checks.

StorageCost: As can be observed from the Table VI, our scheme incurs relatively higher storage overhead due to its multi-replica storage mechanism, and the integration of a fine-grained access control mechanism further introduces additional storage costs. However, this trade-off in overhead fundamentally enhances data availability and integrity—two attributes that are indispensable and critical requirements in consumer data protection scenarios. Ultimately, it achieves an optimal solution that is more aligned with practical application scenarios in the trade-off between storage efficiency and data security.

D. Simulation Experiment

For better performance comparison, we conduct comprehensive experiments to evaluate our scheme under various scenarios.

1) *Local Testbed Configuration:* We implement the cryptographic operations using the pairing-based cryptography (PBC) library in Java, with IntelliJ IDEA 2024 as the

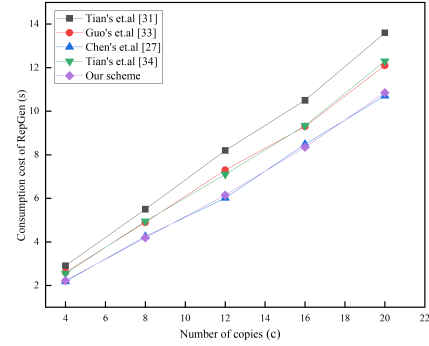


Fig. 2. Time-consumption for the generation of copies.

development environment. The local experiments are conducted on a Lenovo PC running Windows 10, configured with an Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz (boost up to 1.50GHz) and 16GB RAM. For cryptographic primitives, we employ SHA-512 as the hash function, AES-256 for symmetric encryption, and RSA-2048 for asymmetric encryption operations.

2) *Distributed Cloud Environment:* The experiment simulates a typical consumer cloud scenario using a simplified star topology, in which the central management node (one virtual machine) serves as both the GM and the TPA, while the four edge storage nodes (each as one virtual machine) simulate two CSPs and two DOs located in different geographical regions, respectively. Each node features 4 vCPUs and 8GB RAM running Ubuntu 20.04. Using Linux 'tc' and 'netem' tools, four sets of quantitative network impairment parameters were precisely injected into the experimental link: baseline latency (10ms, 20ms, 50ms), random packet loss rate (0.1%, 0.5%, 2.0%), bandwidth limitation (50Mbps, 100Mbps, 500Mbps), and delay jitter (5ms). This systematically simulates diverse conditions ranging from high-quality local area networks to challenging wide area networks, thereby enabling a quantitative evaluation of the performance and robustness of the scheme's core operations.

3) *Parameter Configuration:* We systematically evaluate the scheme with comprehensive parameter settings: data block sizes (4KB–32KB), attribute counts (5–50), replica configurations (3–18 copies), integrity audit challenges (200–1000 blocks), and group scales (50–100 users).

We present comprehensive comparison results in Fig. 2 to 7, covering performance metrics across single-machine efficiency, distributed environment robustness, parameter sensitivity, and real-world application scenarios.

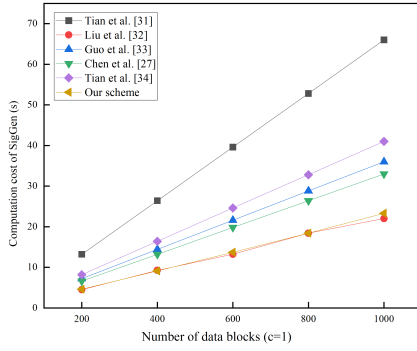


Fig. 3. Time-consumptions for the generations of signatures.

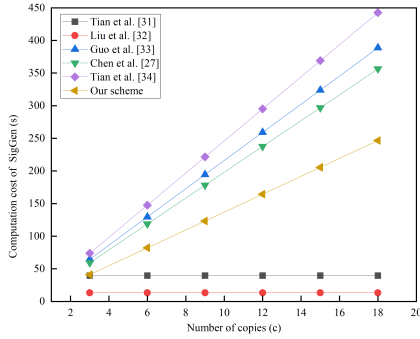


Fig. 4. Time-consumption for the generations of copies' signatures.

As shown in Fig. 2, under the condition that the number of replicas increases from 4 to 20, the replica generation time of all schemes shows a linear growth. It is worth noting that the time overhead of Tian et al.'s scheme [31] is consistently higher than our scheme.

Fig. 3 illustrates the impact of the number of data blocks on the signature generation time. As the number of data blocks increases from 200 to 1000, the signature time of all schemes increases linearly, but there are significant differences in the growth slope. Our scheme demonstrates the optimal linear growth characteristics. Further analysis of the impact of the number of replicas on signature performance (Fig. 4) shows that our scheme maintains low signature overhead even when the number of replicas reaches 18, whereas Tian et al.'s scheme [31], due to its support for only a limited number of replicas, cannot scale its performance with the replica size.

In the verification phase, we focused on the impact of the number of challenge blocks on the verification time. As shown in Fig. 5, the verification time of Guo et al.'s scheme [33] shows an approximately quadratic growth trend. When the number of challenge blocks reaches 100, its verification time is approximately 2.3 times higher than that of our scheme. In contrast, our scheme controls the increase in verification time to a nearly linear range through a carefully designed aggregate verification mechanism, demonstrating excellent scalability.

In Fig. 6, the selected comparative schemes [6] and [15] both involve the decryption of ciphertext data by authorized users. The experiment simulates the time overhead of the decryption algorithms in each scheme under different attribute set sizes to evaluate the efficiency of their access control mechanisms in practical use.

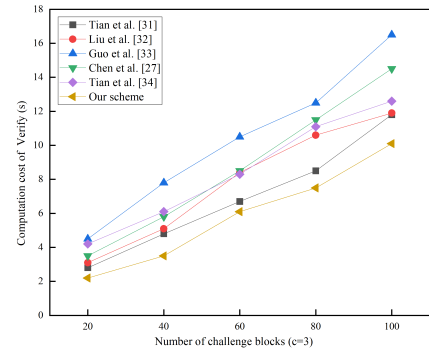


Fig. 5. Time-consumption for the TPA's verification.

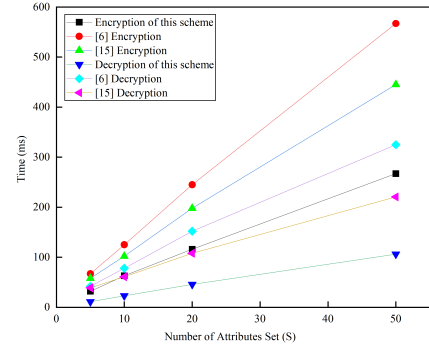


Fig. 6. Time-consumption for the number of attributes.

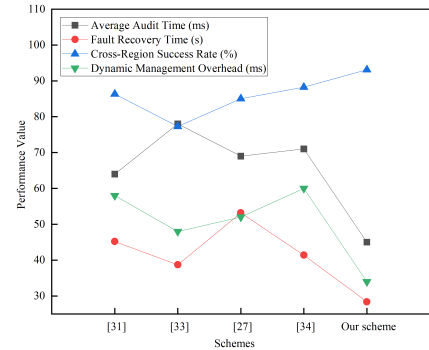


Fig. 7. Performance comparison in distributed environments.

The experimental results indicate that the decryption time of all ABE schemes exhibits a positive correlation with the number of attributes. However, our scheme demonstrates the most gradual increase in time consumption, highlighting its superior computational efficiency. This advantage is attributed to the optimized encryption and decryption mechanisms in our scheme, which effectively reduce the number of required costly operations.

Fig. 7 provides a comprehensive comparison between the proposed scheme and existing works across four key performance metrics in distributed cloud environments.

Our scheme achieves the lowest latency (approximately 32 ms), outperforming the next best scheme by about 15%. This efficiency stems from our optimized aggregate verification mechanism. It reveals that our scheme maintains the highest reliability, significantly exceeding other schemes

TABLE VII
ANALYSIS OF STORAGE-EFFICIENCY TRADE-OFF

Copies (c)	1	2	3	4
Storage (GB)	0.39	0.78	1.17	1.56
Audit delay (ms)	300	334	363	388
Data availability	99.900%	99.999%	99.9999%	99.99999%

by 12-18 percentage points. This validates its robust performance under network partitions. Finally, our scheme introduces minimal latency during user revocation and key updates, showing superior adaptability to organizational changes through our lightweight key update protocol.

From the comparisons of experimental results, our scheme implements more useful features without compromising its performance. Therefore, it is also efficient and can be used in future real-world applications.

E. Storage Efficiency Analysis

To provide a rigorous foundation for evaluating the practical deployment of our multi-copy scheme, this section presents a quantitative analysis of the fundamental trade-off between data availability and storage efficiency.

The total storage overhead S_{total} incurred at the CSP can be formally modeled as follows: $S_{total}(c) = S_{base} + c \cdot (S_{replica} + S_{sign}) + S_{ABE}$. S_{base} denotes fixed base metadata (e.g., system parameters, file headers). $S_{replica}$ is the storage cost of a single replica. Its size is proportional to the original file size $|F|$. S_{sign} is the overhead of the tags. S_{ABE} is the overhead of the access control structure.

Based on the model and experimental profiling, Table VII illustrates the core trade-offs. All calculations are based on the following typical parameter assumptions: $S_{replica} + S_{sign} = 400$ MB (including data and tags), corresponding CP-ABE overhead and base metadata $S_{base} + S_{ABE} = 15KB$, single CSP failure probability $p = 0.01$, base audit verification time for a single replica $T_{base} = 300ms$.

These quantitative analyses demonstrate that the proposed scheme effectively controls the performance overhead introduced by multi-replica storage through aggregated auditing, while the linear storage model enables precise cost prediction. As shown in Table VII, the data availability is calculated based on $A(c) = 1 - p^c$. It can be observed that a configuration of $c = 3$ is recommended as the baseline, as it achieves high availability (99.9999%) while maintaining manageable storage overhead and low audit latency. For extremely critical or frequently accessed data, the value can be increased to $c = 4$ on a case-by-case basis.

VI. CONCLUSION

This paper proposed an efficient multi-copy group data sharing scheme, tailored for consumer IoT environments in cloud storage. Performance comparisons reveal that the scheme achieves an optimal balance between functional completeness and operational efficiency. By centralizing complex computational tasks within the group manager, it significantly reduces

the resource consumption of end-user devices. This scheme is applicable to practical scenarios such as smart homes and connected vehicle networks, demonstrating strong practicality and deployability. Future work will focus on enabling support for fully dynamic data operations by introducing an authenticated data structure, such as a Merkle hash tree, to maintain independent authentication information for each data replica. Subsequent research could concentrate on designing more efficient authentication structures and optimization strategies tailored for multi-replica scenarios, thereby further enhancing the system's scalability in large-scale cloud environments.

REFERENCES

- [1] E. Ahmed et al., "The role of big data analytics in Internet of Things," *Comput. Netw.*, vol. 129, no. 2, pp. 459–471, 2017.
- [2] O. Aouedi et al., "A survey on intelligent Internet of Things: Applications, security, privacy, and future directions," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 2, pp. 1238–1292, Apr. 2025.
- [3] Y. Yang, L. Wu, G. Yin, L. Li, and H. Zhao, "A survey on security and privacy issues in Internet-of-Things," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1250–1258, Oct. 2017.
- [4] P. Sun, S. Shen, Y. Wan, Z. Wu, Z. Fang, and X.-Z. Gao, "A survey of IoT privacy security: Architecture, technology, challenges, and trends," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34567–34591, Nov. 2024.
- [5] H. Tan, "An efficient IoT group association and data sharing mechanism in edge computing paradigm," *Cyber Secur. Appl.*, vol. 1, Dec. 2023, Art. no. 100003.
- [6] G. Yang, P. Li, Y. Xin, Y. He, C. Wang, and X. Chen, "An efficient hierarchical attribute-based encryption scheme with cross-domain data sharing," *Comput. Netw.*, vol. 255, Dec. 2024, Art. no. 110863.
- [7] S. Thouheed Ahmed et al., "Federated learning framework for consumer IoMT-edge resource recommendation under telemedicine services," *IEEE Trans. Consum. Electron.*, vol. 71, no. 1, pp. 252–259, Feb. 2025.
- [8] Z. Xu, X. Chen, L. Chen, X. Lan, H. Ren, and C. Shen, "An efficient and commercial proof of storage scheme supporting dynamic data updates," *Comput. Secur.*, vol. 157, Oct. 2025, Art. no. 104609.
- [9] T. Li, J. Chu, and L. Hu, "CIA: A collaborative integrity auditing scheme for cloud data with multi-replica on multi-cloud storage providers," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 1, pp. 154–162, Jan. 2023.
- [10] H. Tian, F. Nan, H. Jiang, C.-C. Chang, J. Ning, and Y. Huang, "Public auditing for shared cloud data with efficient and secure group management," *Inf. Sci.*, vol. 472, pp. 107–125, Jan. 2019.
- [11] L. Wu, X. Du, M. Guizani, and A. Mohamed, "Access control schemes for implantable medical devices: A survey," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1272–1283, Oct. 2017.
- [12] J. Shen, T. Zhou, X. Chen, J. Li, and W. Susilo, "Anonymous and traceable group data sharing in cloud computing," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 4, pp. 912–925, Apr. 2018.
- [13] Y. Ming, W. Zhang, H. Liu, and C. Wang, "Certificateless public auditing scheme with sensitive information hiding for data sharing in cloud storage," *J. Syst. Archit.*, vol. 143, Oct. 2023, Art. no. 102965.
- [14] J. Qiao, N. Wang, J. Fu, L. Deng, J. Wang, and J. Liu, "A lightweight CP-ABE scheme for EHR over cloud based on blockchain and secure multi-party computation," *Trans. Emerg. Telecommun. Technol.*, vol. 36, no. 2, Feb. 2025, Art. no. e70053.
- [15] J. Li, W. Yao, Y. Zhang, H. Qian, and J. Han, "Flexible and fine-grained attribute-based data storage in cloud computing," *IEEE Trans. Services Comput.*, vol. 10, no. 5, pp. 785–796, Sep. 2017.
- [16] G. Xu, S. Xu, J. Ma, J. Ning, and X. Huang, "An adaptively secure and efficient data sharing system for dynamic user groups in cloud," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 5171–5185, 2023.
- [17] C. Lan, L. Liu, C. Wang, and H. Li, "An efficient and revocable attribute-based data sharing scheme with rich expression and escrow freedom," *Inf. Sci.*, vol. 624, pp. 435–450, May 2023.
- [18] M. Femminella, M. Palmucci, G. Reali, and M. Rengo, "Attribute-based management of secure kubernetes cloud bursting," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 1276–1298, 2024.
- [19] H. Wang, Q. Wu, B. Qin, and J. Domingo-Ferrer, "Identity-based remote data possession checking in public clouds," *IET Inf. Secur.*, vol. 8, no. 2, pp. 114–121, Mar. 2014.

- [20] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, New York, NY, USA, 2007, pp. 598–609.
- [21] A. Juels and B. S. Kaliski, "PORs: Proofs of retrievability for large files," *IACR Cryptol. ePrint Arch.*, vol. 243, pp. 584–597, Jan. 2007.
- [22] H. Yan, J. Li, J. Han, and Y. Zhang, "A novel efficient remote data possession checking protocol in cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 1, pp. 78–88, Jan. 2017.
- [23] C. Wang, S. S. M. Chow, Q. Wang, K. Ren, and W. Lou, "Privacy-preserving public auditing for secure cloud storage," *IEEE Trans. Comput.*, vol. 62, no. 2, pp. 362–375, Feb. 2013.
- [24] H. Tian, N. Gan, F. Peng, H. Quan, C.-C. Chang, and A. V. Vasilakos, "Smart contract-based public integrity auditing for cloud storage against malicious auditors," *Future Gener. Comput. Syst.*, vol. 166, May 2025, Art. no. 107709.
- [25] J. Li, H. Yan, and Y. Zhang, "Certificateless public integrity checking of group shared data on cloud storage," *IEEE Trans. Services Comput.*, vol. 14, no. 1, pp. 71–81, Jan. 2021.
- [26] J. Li, H. Yan, and Y. Zhang, "Identity-based privacy preserving remote data integrity checking for cloud storage," *IEEE Syst. J.*, vol. 15, no. 1, pp. 577–585, Mar. 2021.
- [27] G. Chen, R. Hao, and M. Yang, "Popularity-based multiple-replica cloud storage integrity auditing for big data," *Future Gener. Comput. Syst.*, vol. 163, Feb. 2025, Art. no. 107534.
- [28] L. Zhou, A. Fu, G. Yang, H. Wang, and Y. Zhang, "Efficient certificateless multi-copy integrity auditing scheme supporting data dynamics," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1118–1132, Mar. 2022.
- [29] J. Li, H. Yan, and Y. Zhang, "Efficient identity-based provable multi-copy data possession in multi-cloud storage," *IEEE Trans. Cloud Comput.*, vol. 10, no. 1, pp. 356–365, Jan. 2022.
- [30] Z. Tu, X. An Wang, W. Du, Z. Wang, and M. Lv, "An improved multi-copy cloud data auditing scheme and its application," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 35, no. 3, pp. 120–130, Mar. 2023.
- [31] Y. Tian, H. Tan, J. Shen, V. Pandi, B. B. Gupta, and V. Arya, "Efficient identity-based multi-copy data sharing auditing scheme with decentralized trust management," *Inf. Sci.*, vol. 644, Oct. 2023, Art. no. 119255.
- [32] Z. Liu, S. Wang, and Y. Liu, "Blockchain-based integrity auditing for shared data in cloud storage with file prediction," *Comput. Netw.*, vol. 236, Nov. 2023, Art. no. 110040.
- [33] Z. Guo, K. Zhang, L. Wei, S. Chen, and L. Wang, "RDIMM: Revocable and dynamic identity-based multi-copy data auditing for multi-cloud storage," *J. Syst. Archit.*, vol. 141, Aug. 2023, Art. no. 102913.
- [34] H. Tian, M. Wang, H. Quan, C.-C. Chang, and A. V. Vasilakos, "TEEMRDA: Leveraging trusted execution environments for multi-replica data auditing in cloud storage," *Comput. Secur.*, vol. 150, Mar. 2025, Art. no. 104250.



Jinyong Chang received the B.S. degree in mathematics from Shanxi University in 2004, the M.S. degree in mathematics from Capital Normal University in 2009, and the Ph.D. degree in information security from SKLOIS, Institute of Information Engineering, Chinese Academy of Sciences, in 2015. He was a Post-Doctoral Researcher with Peking University, Beijing, China. Currently, he is a Professor with the School of Information and Control, XAUAT. His research interests mainly focus on cryptography and information security.

Ru Meng is with the College of Cyber Security, Jinan University, Guangzhou, China.



Peiru Yang was born in 2000. She is currently pursuing the master's degree with the School of Information and Control Engineering, Xi'an University of Architecture and Technology (XAUAT), Xi'an, Shaanxi, China. Her research interests include information security and cryptography.



Youtong Wang was born in 2000. He is currently pursuing the master's degree with the School of Information and Control Engineering, Xi'an University of Architecture and Technology (XAUAT), Xi'an, Shaanxi, China. His research interests include information security and cryptography.



Xia Zhou was born in 2000. She is currently pursuing the master's degree with the School of Information and Control Engineering, Xi'an University of Architecture and Technology (XAUAT), Xi'an, Shaanxi, China. Her research interests include information security and cryptography.